

LAB # 03b

Implementation of various operators in java

Objective

To get familiarity with the various operators in java.

Lab Goals

To provide working insight into the usage of various operators in java programming:

- a. Demonstrate the implementation of various operators.
- b. Explore Arithmetic, Assignment, Relation, Logical, Ternary and Bitwise operators in Java.
- c. Explore Auto-increment and Auto-decrement Operators

Introduction to operators

In Java, operators are symbols used to perform operations on variables and values. They are categorized into different types based on the operations they perform, such as arithmetic, relational, logical, bitwise, assignment, and more.

Basic Arithmetic Operators

These operators are used to perform basic arithmetic operations like addition, subtraction, multiplication, division, and modulus.

- + (Addition)
- - (Subtraction)
- * (Multiplication)
- / (Division)
- % (Modulus)

Example

```
public class Main {  
    public static void main(String[] args) {  
  
        // Arithmetic Operators  
        int a = 10;  
        int b = 5;  
        int sum = a + b;    // Addition  
        int difference = a - b; // Subtraction  
        int product = a * b; // Multiplication  
        int quotient = a / b; // Division  
        int remainder = a % b; // Modulus  
    }  
}
```

```
// Print the results
System.out.println("***** Arithmetic Operators *****\n");
System.out.println("The Sum: " + sum);
System.out.println("The Difference: " + difference);
System.out.println("The Product: " + product);
System.out.println("The Quotient: " + quotient);
System.out.println("The Remainder: " + remainder);

}
}
```

Output

```
***** Arithmetic Operators *****

The Sum: 15
The Difference: 5
The Product: 50
The Quotient: 2
The Remainder: 0
```

Assignment Operators

These operators are used to assign values to variables.

- = (Simple assignment)
- +=, -=, *=, /=, %= (Compound assignment)

Example

```
public class Main {
    public static void main(String[] args) {

        // Assignment Operators
        int num = 131;
        num += 5; // num = num + 5;
        num -= 3; // num = num - 3;
        num *= 2; // num = num * 2;
        num /= 4; // num = num / 4;
        num %= 39; // num = num % 3;

        // Print the results
        System.out.println("***** Assignment Operators *****\n");
        System.out.println("num after operations: " + num);

    }
}
```

Output

```
***** Assignment Operators *****  
  
num after operations: 27
```

Relational Operators

These operators are used to compare values and determine the relationship between them. They return a Boolean value (**true or false**).

- **== (Equal to)**
- **!= (Not equal to)**
- **> (Greater than)**
- **< (Less than)**
- **>= (Greater than or equal to)**
- **<= (Less than or equal to)**

Example

```
public class Main {  
    public static void main(String[] args) {  
  
        int a = 10;  
        int b = 5;  
  
        // Relational Operators  
        boolean isEqual = (a == b);    // Equal to  
        boolean notEqual = (a != b);   // Not equal to  
        boolean greaterThan = (a > b);  // Greater than  
        boolean lessThan = (a < b);     // Less than  
        boolean greaterOrEqual = (a >= b); // Greater than or equal to  
        boolean lessOrEqual = (a <= b);  // Less than or equal to  
  
        // Print the results  
        System.out.println("***** Relational Operators *****\n");  
        System.out.println("Is Equal: " + isEqual);  
        System.out.println("Not Equal: " + notEqual);  
        System.out.println("Greater Than: " + greaterThan);  
        System.out.println("Less Than: " + lessThan);  
        System.out.println("Greater or Equal: " + greaterOrEqual);  
        System.out.println("Less or Equal: " + lessOrEqual);  
  
    }  
}
```

Output

```
***** Relational Operators *****

Is Equal: false
Not Equal: true
Greater Than: true
Less Than: false
Greater or Equal: true
Less or Equal: false
```

Logical Operators

These operators are used to perform logical operations on Boolean values.

- **&& (Logical AND)**
- **|| (Logical OR)**
- **! (Logical NOT)**

Example

```
public class Main {
    public static void main(String[] args) {

        // Logical Operators
        boolean andResult = (true && false); // Logical AND
        boolean orResult = (true || false); // Logical OR
        boolean notResult = !true;          // Logical NOT

        // Print the results
        System.out.println("***** Logical Operators *****\n");
        System.out.println("AND Result: " + andResult);
        System.out.println("OR Result: " + orResult);
        System.out.println("NOT Result: " + notResult);
    }
}
```

Output

```
***** Logical Operators *****

AND Result: false
OR Result: true
NOT Result: false
```

Bitwise Operators

These operators perform bitwise operations on integer operands.

- **& (Bitwise AND)**
- **| (Bitwise OR)**
- **^ (Bitwise XOR)**
- **~ (Bitwise Complement)**
- **<< (Left shift)**
- **>> (Signed right shift)**

Example

```
public class Main {
    public static void main(String[] args) {

        // Bitwise Operators
        int x = 5; // 101 in binary
        int y = 3; // 011 in binary
        int bitwiseAnd = x & y; // Bitwise AND
        int bitwiseOr = x | y; // Bitwise OR
        int bitwiseXor = x ^ y; // Bitwise XOR
        int bitwiseComplement = ~x; // Bitwise Complement
        int leftShift = x << 1; // Left Shift
        int rightShift = x >> 1; // Right Shift

        // Print the results
        System.out.println("***** Bitwise Operators *****\n");
        System.out.println("Bitwise AND: " + bitwiseAnd);
        System.out.println("Bitwise OR: " + bitwiseOr);
        System.out.println("Bitwise XOR: " + bitwiseXor);
        System.out.println("Bitwise Complement: " + bitwiseComplement);
        System.out.println("Left Shift: " + leftShift);
        System.out.println("Right Shift: " + rightShift);
    }
}
```

Output

```
***** Bitwise Operators *****

Bitwise AND: 1
Bitwise OR: 7
Bitwise XOR: 6
Bitwise Complement: -6
Left Shift: 10
Right Shift: 2
```

```
num1 = 11; /* equal to 00001011*/  
num2 = 22; /* equal to 00010110 */
```

Bitwise operator performs bit by bit processing.

num1 & num2 compares corresponding bits of num1 and num2 and generates 1 if both bits are equal, else it returns 0. In our case it would return: 2 which is 00000010 because in the binary form of num1 and num2 only second last bits are matching.

num1 | num2 compares corresponding bits of num1 and num2 and generates 1 if either bit is 1, else it returns 0. In our case it would return 31 which is 00011111

num1 ^ num2 compares corresponding bits of num1 and num2 and generates 1 if they are not equal, else it returns 0. In our example it would return 29 which is equivalent to 00011101

~num1 is a complement operator that just changes the bit from 0 to 1 and 1 to 0. In our example it would return -12 which is signed 8 bit equivalent to 11110100

num1 << 2 is left shift operator that moves the bits to the left, discards the far left bit, and assigns the rightmost bit a value of 0. In our case output is 44 which is equivalent to 00101100

Note: In the example below we are providing 2 at the right side of this shift operator that is the reason bits are moving two places to the left side. We can change this number and bits would be moved by the number of bits specified on the right side of the operator. Same applies to the right side operator.

num1 >> 2 is right shift operator that moves the bits to the right, discards the far right bit, and assigns the leftmost bit a value of 0. In our case output is 2 which is equivalent to 00000010

Auto-increment (++) Operator

- **Prefix (++var):** When used as a prefix, the value of the variable is incremented by 1, and then the incremented value is used in the expression.
- **Postfix (var++):** When used as a postfix, the current value of the variable is used in the expression, and then the value is incremented by 1.

Example

```
public class Main {  
    public static void main(String[] args) {  
  
        // Auto-increment operator (++)  
        int x = 5;  
        System.out.println("\tOriginal value of x: " + x + "\n");  
  
        // Prefix increment: increments x first, then assigns the incremented value to y  
        int y = ++x;  
  
        // Print the results  
        System.out.println("***** Prefix increment *****");  
        System.out.println("Prefix Increment: x = " + x + ", y = " + y);  
  
        // Postfix increment: assigns the current value of x to z, then increments x  
        int z = x++;  
  
        // Print the results  
        System.out.println("\n***** Postfix increment *****");  
        System.out.println("Postfix Increment: x = " + x + ", z = " + z);  
    }  
}
```

Output

```
Original value of x: 5  
  
***** Prefix increment *****  
Prefix Increment: x = 6, y = 6  
  
***** Postfix increment *****  
Postfix Increment: x = 7, z = 6
```

Auto-decrement (--) Operator

- **Prefix (--var):** When used as a prefix, the value of the variable is decremented by 1, and then the decremented value is used in the expression.
- **Postfix (var--):** When used as a postfix, the current value of the variable is used in the expression, and then the value is decremented by 1.

Example

```
public class Main {  
    public static void main(String[] args) {  
  
        // Auto-decrement operator (--)  
        int a = 10;  
        System.out.println("\tOriginal value of a: " + a + "\n");  
  
        // Prefix decrement: decrements a first, then assigns the decremented value to b  
        int b = --a;  
  
        // Print the results  
        System.out.println("***** Prefix Decrement *****");  
        System.out.println("Prefix Decrement: a = " + a + ", b = " + b);  
  
        // Postfix decrement: assigns the current value of a to c, then decrements a  
        int c = a--;  
  
        // Print the results  
        System.out.println("\n***** Postfix Decrement *****");  
        System.out.println("Postfix Decrement: a = " + a + ", c = " + c);  
  
    }  
}
```

Output

```
Original value of a: 10  
  
***** Prefix Decrement *****  
Prefix Decrement: a = 9, b = 9  
  
***** Postfix Decrement *****  
Postfix Decrement: a = 8, c = 9
```

Conditional Operator (Ternary Operator)

This operator evaluates a Boolean expression and returns one of two values depending on whether the expression is true or false.

- **? : (Conditional operator)**

Syntax: variable num1 = (expression) ? value *if true* : value *if false*

Example

```
public class Main {  
    public static void main(String[] args) {  
  
        int a = 10;  
        int b = 20;  
        int max = (a > b) ? a : b; // If a is greater than b, assign a to max, otherwise assign b.  
  
        // Print the results  
        System.out.println("\n***** Ternary Operator *****");  
        System.out.println("The maximum value is: " + max);  
  
    }  
}
```

Output

```
***** Ternary Operator *****  
The maximum value is: 20
```

Another Example

```
public class OperatorDemo {
    public static void main(String args[]) {
        int num1 = 100;
        int num2 = 20;

        System.out.println("num1 + num2: " + (num1 + num2) );
        num2 = num1;
        System.out.println("= Output: "+num2);
        num1++;
        num2--;
        System.out.println("num1++ is: "+num1);
        System.out.println("num2-- is: "+num2);
        boolean b1 = true;
        boolean b2 = false;

        System.out.println("b1 && b2: " + (b1&&b2));
        int num1 =11; /* 11 = 00001011 */
        int num2 =22; /* 22 = 00010110 */
        int result =0;

        result = num1 & num2;
        System.out.println("num1 & num2: "+result);
        int num1, num2;
        num1 =25;
        /* num1 is not equal to 10 that's why
        * the second value after colon is assigned
        * to the variable num2
        */
        num2 =(num1 ==10)?100:200;
        System.out.println("num2: "+num2);
    }
}
```

Exercise

- A. Create a Java application for a basic calculator with the following capabilities: Addition and Subtraction, Division, Exponentiation, Multiplication, also use bitwise operator and Ternary operator to find the maximum, minimum value.
- B. Create a program that demonstrates operator precedence by showing how parentheses can change the result of complex expressions.

Hint:

- Initialize variables with values that will clearly show precedence differences
int a = 10, b = 5, c = 2, d = 3;
- **Expression 1:** Without parentheses
- **Expression 2:** With parentheses changing the order

- C. Write a program that uses bitwise operators to swap two numbers without using a temporary variable.

Hint:

- Swap using **XOR** bitwise operator

- D. To convert temperature from Celsius to Fahrenheit, write a Java program

Guidelines:

Asking for the temperature in Celsius will prompt the user.

Check the temperature that the user has entered.

Use this formula to convert the temperature from Celsius to Fahrenheit: Degrees Celsius * 9/5) + 32 = Fahrenheit

Print the Fahrenheit temperature conversion.

- E. Write a program which shows the implementation of increment and decrement operators.
- F. Write a program which shows the implementation of Ternary Operator.

Conditions:

- Check even and odd number.
- Check Absolute value.